Algoritmos y Estructuras de Datos
2do Parcial – Miércoles 19 de noviembre de 2025

Libreta	Apellido y Nombre	E1	E2	E3	E4	E5	Nota Final	Corrigió

E1. Diseño (25 pts)

Se quiere implementar un sistema para una empresa de reparto de paquetes. El sistema tiene un conjunto de repartidores, y cada repartidor se identifica con un código alfanumérico de 10 caracteres. Los pedidos de entregas se reciben a través de una app, y se les asigna un código de identificación (entero no acotado). Como la asignación de precios es dinámica, cada pedido puede tener un precio distinto.

Cuando un repartidor solicita que se le asigne una entrega, siempre quiere la entrega más rentable, por lo que el sistema directamente deberá asignarle aquella que tenga el mayor precio por kilómetro (considerando el precio de la entrega y la distancia a la base de operaciones).

Se desea conocer, para cada repartidor, la ganancia total entre todos los viajes que realizó. Además nos interesa saber el precio por kilómetro promedio entre todos los viajes realizados por todos los repartidores (sin considerar los viajes pendientes).

Nota: si no se indica **Requiere** o **Complejidad**, el primero es $True$ y la segunda puede ser cualquiera

Monto `ES` $\mathbb{R}$	Repartidor `ES` string (max: 10 caracteres)
Distancia `ES` $\mathbb{Z}$ (en kms)	Entrega `ES` $\mathbb{Z}$ (no acotado)

- `proc NuevoSistema`(in $r : Conjunto(Repartidor)$) : $Sistema$
  - **Descripción**: Inicializa el sistema de repartos
- `proc NuevaEntrega`(inout $s : Sistema$, in $e : Entrega$, in $precio : Monto$, in $dist : Distancia$)
  - **Descripción**: Registra una nueva entrega en el sistema, que quedará pendiente hasta que se le asigne un repartidor. Se pasa el precio que el cliente va a pagar y la distancia a la base de operaciones.
  - **Requiere**: Monto y distancia son positivos.
- `proc AsignarEntrega`(inout $s : Sistema$, in $r : Repartidor$)
  - **Descripción**: Se le asigna al repartidor r la entrega pendiente que tiene mayor precio por km.
  - **Requiere**: r pertenece a los repartidores registrados en el sistema y hay al menos una entrega pendiente.
  - **Complejidad**: $O(log\ e)$ donde e es la cantidad de entregas pendientes.
- `proc PrecioPorKmPromedio`(in $s : Sistema$) : $Monto$
  - **Descripción**: Devuelve el precio por km promedio (entre todos los viajes realizados por todos los repartidores, sin contar los viajes pendientes)
  - **Complejidad**: $O(1)$
- `proc GananciaDeRepartidor`(in $s : Sistema$, in $r : Repartidor$) : $Monto$
  - **Descripción**: Devuelve la ganancia total obtenida hasta el momento por el repartidor r
  - **Requiere**: El repartidor r pertenece a los repartidores registrados en el sistema
  - **Complejidad**: $O(1)$

Se pide:

a) Proponga una estructura de representación para implementar el módulo Sistema. Describa con palabras qué hace cada operación y justifique por qué la estructura propuesta cumple con las complejidades requeridas.
b) Escriba en pseudocódigo el algoritmo para la operación **AsignarEntrega**

E2. Preguntas teóricas (10 pts)

Elegir la opción correcta **sin justificar** su respuesta.

a) *Diseño*: Indique si las siguientes frases son verdaderas o falsas
   i) el invariante de representación puede no ser válido entre dos instrucciones de un proc
   ii) el invariante de representación se tiene que cumplir al finalizar un proc pero no es necesario que se cumpla inmediatamente antes de ejecutar el proc.
   iii) el invariante de representación no puede ser *true*

b) *Estructuras*: Considerar una variante de heaps en la que cada nodo tiene d hijos, con $d > 2$. Marcar verdadero o falso:
   i) La altura es lineal
   ii) La altura es menor o igual que en el caso de los heaps binarios
   iii) No pueden ser izquierdistas
   iv) Se pueden utilizar para los mismos fines que los heaps binarios, pero con mayor complejidad asintótica en el peor caso.
   v) Se pueden utilizar para los mismos fines que los heaps binarios, con igual complejidad asintótica.
   vi) No se pueden utilizar para los mismos fines que los heaps binarios pues no se puede reconocer el mínimo elemento.

c) *Sorting*: Describa brevemente qué quiere decir que un algoritmo de ordenamiento sea estable

E3. Complejidad (20 pts)

a. Indique si las siguientes afirmaciones son verdaderas o falsas. No es necesario justificar.
   i. si $f + g \in \Omega(h)$ entonces $f \in \Omega(h)$
   ii. si el peor caso de un algoritmo es $\Omega(n)$ entonces no puede pasar que su mejor caso sea $O(n^2)$

b. Determinar y explicar con palabras la complejidad del peor caso de los siguientes algoritmos.

```
1  func sumaCubos(in n : int):
2      i ← 1;
3      res ← 0;
4      while i < n^2 do
5          fact ← 1;
6          j ← 0;
7          while j < 3 do
8              fact ← fact * i;
9              j ← j + 1;
10         res ← res + fact;
11         i ← i + 1;
12     return res;
```

```
1  func sumaPotenciasDe2(in n : int):
2      i ← 1;
3      res ← 0;
4      while i < n do
5          res ← res + i;
6          i ← i * 2;
7      return res;
```

E4. Rep&Abs (20 pts)

Se tiene un TAD que modela una biblioteca. La biblioteca tiene una serie de estantes, numerados del 0 al n-1. En cada estante se guarda un conjunto de libros. Cuando se presta un libro, se retira del estante y se registra a quién se lo prestó y la fecha de devolución. Cuando se devuelve, se vuelve a poner en algún estante.

Libro es int *Persona* es string *Fecha* es int

TAD Biblioteca {

// Secuencia de estantes. Cada estante es una secuencia de libros.
obs estantes : seq⟨seq⟨*Libro*⟩⟩

// Dado un libro que está prestado, indica quién lo tiene y cuándo deberá devolverlo.
obs prestamos : dicc⟨*Libro*, tupla ⟨*Persona*, *Fecha*⟩⟩

}

Este TAD se modeló con la siguiente estructura:

Módulo BibliotecaImpl **implementa** Biblioteca <

```
// Cantidad de estantes
var cantEstantes: int
// Estante en el que está ubicado cada libro (si no está prestado)
var ubicación: Diccionario<Libro, int>
// Si el libro está prestado, este diccionario indica quién lo pidió
var prestamoQuien: Diccionario<Libro, Persona>
// Fecha de devolución de un libro (si está prestado)
var prestamoFecha: Diccionario<Libro, Fecha>
// Dada una fecha, todos los libros que tienen que ser devueltos en esa fecha
var venceFecha: Diccionario<Fecha, Conjunto<Libro>>
```

>

a) Indique si cada una de las sentencias propuestas pertenecen al invariante de representación para la estructura propuesta
   i) Las claves de **ubicacion**, **prestamoQuien**, **prestamoFecha** son las mismas
   ii) Todos los valores de **ubicación** son enteros entre 0 y **cantEstantes**-1
   iii) Las claves de **prestamoQuien** son todas distintas
   iv) Si un libro está en **ubicación** entonces no está en **prestamoQuien**
   v) Todos los libros de los valores de **venceFecha** son claves de **prestamoQuien**.
   vi) Los valores de **prestamoQuien** son todos distintos
b) Escriba la función de abstracción en lógica de primer orden

E5. Sorting (25 pts)

En un supermercado los productos se identifican con un código de 10 dígitos que tiene la siguiente forma: **"MMMPPPPPPP"**, donde se reservan 3 dígitos para la marca (**M**) y 7 dígitos para el producto (**P**). Periódicamente se recibe una lista de aumentos que contiene, para cada marca, el porcentaje de aumento que debe aplicarse a todos los productos de dicha marca.

Dada una secuencia de códigos de productos con el precio de cada uno (ordenada por precio de menor a mayor) y una lista de aumentos, se quiere actualizar la secuencia de productos, aplicando el aumento que le corresponde a cada uno, y devolver dicha lista ordenada de menor a mayor por el nuevo precio. En caso de empate, se desea utilizar para desempatar el código en orden lexicográfico (como figura en el diccionario).

Escriba en pseudocódigo un algoritmo que haga esto en $O(n \log n)$, donde n es la cantidad total de productos. Justifique las decisiones que toma y la complejidad del algoritmo.

Se puede asumir que existe una función `proc substring`(in s : *string*, in *desde* : *int*, in *hasta* : *int*) : *string* que devuelve un nuevo string que es igual al string original entre las posiciones *desde* y *hasta*. La complejidad de dicha función es $O(k)$ donde k es la longitud del string de entrada.

Ejemplo

Entrada:

```
Productos:
    <"RCZDGFLRTY", 800>
    <"AAXRGSDGSA", 1000>
    <"XAQGFKYTAH", 1760>
    <"AAXGAFDHDF", 2400>

Aumentos:
    <"AAX", 0.1>
    <"RCZ", 0.9>
    <"XAQ", 0.5>
```

```
Salida:
    <"AAXRGSDGSA", 1100>
    <"RCZDGFLRTY", 1520>
    <"AAXGAFDHDF", 2640>
    <"XAQGFKYTAH", 2640>
```